第二次课程作业

作业内容：把 `Assignment2` 中给出的 C# 示例代码逐个运行，并结合运行结果进行说明。

测试环境（Windows）

本作业在 **Windows** 下完成，环境如下。

项目	说明
操作系统	Windows 10/11（本机版本：`10.0.26200`）
.NET SDK	`9.0.301`
目标框架	`net9.0`
运行命令	`dotnet run`

---

0 运行准备

统一运行方式：

```
cd Assignment2
dotnet new console --force
dotnet run
```

---

1 委托与事件

输出内容：

```
D:\GitHub\ycc\Dotnet_Course_Assignment\Assignment2\01 委托与事件>dotnet run
Morning: Fred
早上好: Fred
```

说明：

- 代码定义了委托 `GreetingDelegate` 与接口 `IGreeting`。
- `Main()` 当前调用 `TestInterface()`，通过多态分别执行 `EnglishGreeting_cls` 与 `ChineseGreeting_cls` 的实现。
- 结果展示了“同一调用入口，不同实现行为”的接口分派效果。

---

2 返回委托与闭包

输出内容：

```
D:\GitHub\ycc\Dotnet_Course_Assignment\Assignment2\02 返回委托与闭包>dotnet run
返回函数,使用外部变量
r1: 1503  r2: 2503
r1: 2003  r2: 3503
```

说明：

- `someFunc(int one)` 返回一个 `Func<int,int,int>`，并捕获外部局部变量 `two`。
- `fnOne = someFunc(500)` 与 `fnTwo = someFunc(1000)` 各自持有独立闭包状态。
- 第二次调用时，`two` 在各自闭包中继续累加，因此 `r1/r2` 都发生变化，验证了闭包“记住环境”的特性。

---

3 逆变与协变

输出内容：

```
D:\GitHub\ycc\Dotnet_Course_Assignment\Assignment2\03 逆变与协变>dotnet run
Hi
In DerivedClass vf
In BaseClass vf_new
In DerivedClass vf_abstract
In BaseClass vf_sealed
In DerivedClass vf
In DerivedClass vf_new
In DerivedClass vf_abstract
In BaseClass vf_sealed
```

说明：

- `o` 的静态类型是 `BaseClass`，运行时对象是 `DerivedClass`。
- `vf()` 为 `override`，因此发生虚调用，执行派生类实现。
- `vf_new()` 使用 `new` 隐藏而非重写：通过 `BaseClass` 引用调用时执行基类版本，通过 `DerivedClass` 引用调用时执行派生类版本。
- `vf_abstract()` 在派生类完成重写后正常分派。
- `ToString()` 在基类被 `sealed override`，派生类不能再重写，调用均落到基类实现。

---

4 异常处理

4.1 ThrowingAnException

输出内容：

```
D:\GitHub\ycc\Dotnet_Course_Assignment\Assignment2\04 异常\01 ThrowingAnException>dotnet run
ArgumentException: System.ArgumentException: The argument did not represent a digit (Parameter 'textDigit')
   at Program.Parse(String textDigit) in D:\GitHub\ycc\Dotnet_Course_Assignment\Assignment2\04 异常\01 ThrowingAnException\Prog
ram.cs:line 19
   at Program.Main() in D:\GitHub\ycc\Dotnet_Course_Assignment\Assignment2\04 异常\01 ThrowingAnException\Program.cs:line 29
```

说明：

- `Parse("9")` 会失败，因为数组中是英文单词（`"nine"`）而不是字符数字。
- 代码抛出 `ArgumentException`，并用 `nameof(textDigit)` 标注参数名。
- `catch(ArgumentException)` 命中，体现“先捕获具体异常，再捕获通用异常”的写法。

4.2 CatchingDifferentExceptionTypes

输出内容：

```
D:\GitHub\ycc\Dotnet_Course_Assignment\Assignment2\04 异常\02 CatchingDifferentExceptionTypes>dotnet run
D:\GitHub\ycc\Dotnet_Course_Assignment\Assignment2\04 异常\02 CatchingDifferentExceptionTypes\Program.cs(34,33): warning CS8600
: 将 null 文本或可能的 null 值转换为不可为 null 类型。
Win32Exception: -2147467259
In Finally
```

说明：

- 示例抛出 `new Win32Exception(5)`。
- 首个 `catch(Win32Exception) when (...)` 带过滤器，条件成立后命中该分支。
- `finally` 总会执行，因此固定输出 `In Finally`。

4.3 UsingExceptionDispatchInfoToRethrowException

输出内容：

```
D:\GitHub\ycc\Dotnet_Course_Assignment\Assignment2\04 异常\03 UsingExceptionDispatchInfoToRethrowException\Program.cs(25,33):
 warning SYSLIB0014: “WebRequest.Create(string)”已过时:“WebRequest, HttpWebRequest, ServicePoint, and WebClient are obsolete.
 Use HttpClient instead.” (https://aka.ms/dotnet-warnings/SYSLIB0014)
http://www.baidu.com
---1--- TID: 1 InPool: False
.---2--- TID: 8 InPool: True
---3--- TID: 8 InPool: True

Result 2.23 KB
-----------End 1------------
-----------End 2------------
```

说明：

- 演示 `await` 前后线程上下文变化（主线程到线程池线程）。
- 主线程通过 `task.Wait(100)` 轮询并打印 `.`，体现异步任务进行中。
- 示例中请求成功并输出网页文本大小；若发生异常，会在 `AggregateException` 中被 `ExceptionDispatchInfo.Capture(...).Throw()` 重新抛出且尽量保留原始栈信息。

4.4 CreatingCustomException

输出内容：

```
D:\GitHub\ycc\Dotnet_Course_Assignment\Assignment2\04 异常\04 CreatingCustomException>dotnet run
Unhandled exception. DatabaseException: Error
   at DatabaseException.Main() in D:\GitHub\ycc\Dotnet_Course_Assignment\Assignment2\04 异常\04 CreatingCustomException\Progr
am.cs:line 45
```

说明：

- 代码定义了自定义异常 `DatabaseException`，并在 `Main()` 中直接 `throw new DatabaseException("Error")`。
- 由于未做 `try-catch`，所以进程以“未处理异常”方式结束，这是该示例用于演示自定义异常抛出的预期现象。

4.5 check unchecked

输出内容

```
D:\GitHub\ycc\Dotnet_Course_Assignment\Assignment2\04 异常\05 check unchecked>dotnet run
MaxValue: 2147483647
MaxValue+1: -2147483648
```

说明：

- 当前启用的是未显式 `checked` 的加法，`int` 溢出后发生回绕（补码截断），得到最小值。
- 若改为 `checked` 代码块（文件里已给出注释版本），同样操作会抛出 `OverflowException`。

---

5 总结

本次作业完整跑通了委托、闭包、继承多态、异常处理与整数溢出等示例，关键结论如下：

1. 委托与接口都能实现“行为传递”，接口更偏向对象多态。
2. 闭包会捕获并保存外部变量状态，不同返回委托各自维护独立状态。
3. `override` 与 `new` 的差异会直接体现在运行时分派结果上。
4. 异常处理中“具体异常优先、过滤器、finally 保底执行”是推荐模式。
5. `checked/unchecked` 决定了整数溢出是抛错还是回绕。

对异常的认知

异常是程序在运行期对“非预期情况”的结构化反馈机制，不是普通流程控制工具。实践中应优先抛出语义明确的具体异常类型，并在合适边界进行分层捕获：底层保证资源释放与上下文保留，中上层决定重试、降级或终止。相比简单返回错误码，异常能携带调用栈与上下文信息，更利于定位问题；但也应避免吞掉异常或过度捕获 `Exception`，否则会掩盖真实故障。一个稳健的系统应将“正常路径”与“异常路径”清晰分离，在 `finally` 中做清理，在日志中记录关键信息，并让异常成为提升代码可靠性与可维护性的手段。